

**Drone Target Identification Model:
Dataset Builder, Multi-Output CNN, and YOLO-Based Utilization Program
Logan Edwards
Deep Learning (4320)
December 6, 2025**

1. Introduction

This project investigates an end-to-end deep learning pipeline for drone- and camera-based target identification. The overall goal is to design and implement a system that can ingest diverse image datasets, train a classification model that distinguishes valid targets from non-targets and background, and deploy the trained model in a real-time setting using a YOLO-based detection or segmentation front end. The work emphasizes practical concerns such as dataset unification, label consistency, computational efficiency, and the ability to integrate the classifier into a streaming video application that could be used with drone footage or fixed surveillance cameras.

The project is structured around three main components:

- A dataset builder that downloads, reorganizes and standardizes images and labels from multiple public sources.
- A multi-output convolutional neural network (CNN) that predicts both a fine-grained object class and a coarse three-way validity label.
- A real-time utilization program that combines YOLO detections or segmentations with the trained classifier to highlight valid targets in live or preprocessed video.

Together, these components form a modular pipeline that can be adapted to different target definitions or operational contexts while remaining computationally practical for real-time use.

2. Data and Image Builder

The starting point for the project is the construction of a coherent training dataset. Publicly available image datasets that contain people, vehicles or related objects are often organized with different directory structures, label conventions and metadata formats. To address this, a general-purpose unified dataset builder script is implemented. This script downloads one or more source datasets using KaggleHub and restructures them into a single directory called images, accompanied by a metadata file called metadata.csv.

For each source dataset, the builder applies a dataset-specific label extraction strategy, such as:

- Reading labels from folder names (for example, *valid*, *background* or other semantic categories).
- Parsing labels from CSV files using columns such as `label` or `class_name`.
- Inferring labels from filenames using regular expressions when necessary.

The aim is to convert all of these heterogeneous sources into a single, consistent set of image paths and textual labels. Because different datasets may use slightly different terminology for similar concepts, a global label aliasing mechanism is introduced. For example, one dataset might use “pedestrian,” another might use “person,” and a third might label images as “walker.” The dataset builder maps all of these terms onto a canonical fine label such as `person_pedestrian`. Similarly, terms like “background,” “none” or “no_target” are aliased into a single fine label that represents pure background.

Each fine label is further mapped onto one of three coarse labels:

- **valid** – the actual targets of interest for the system.
- **invalid_nontarget** – real objects that are not considered valid targets but might be visually confusing, such as bystanders or vehicles when they are not the designated target.
- **invalid_background** – images that contain no meaningful object or that are considered junk data.

Once labels are unified, the builder writes a row into `metadata.csv` for each image. Each row includes at least four fields: the relative output path under the images directory (`out_relpath`), the canonical fine label (`fine_label`), the coarse validity label (`coarse_label`), and a split indicator that marks the example as belonging to the training, validation or test set. Additional fields may be included as needed. The builder is capable of performing hash-based de-duplication, which detects when the same image appears in multiple datasets and ensures that it is only counted once in the unified corpus.

The dataset is split into training, validation and test subsets, typically with proportions such as 70 %, 15 % and 15 %, respectively. The splitting procedure can be stratified on both the fine and coarse labels so that each subset contains a representative sample of both valid and invalid categories. Optional balancing strategies are also supported, including class-balanced sampling across fine labels and caps on the number of examples per label to prevent extreme dominance by common invalid classes. A set of quality checks verifies that all listed image files exist on disk, that corrupted images are either skipped or logged, and that each split contains at least a minimum number of valid examples. The final dataset bundle consists of the images directory, the `metadata.csv` file and an optional `label_map.json` file that records the mapping from label strings to integer indices.

3. Model Architecture

The classifier used in this project is a compact, regularized convolutional neural network with two output heads. The first head predicts the fine-grained class label, while the second head predicts the three-way coarse validity label. This multi-output design encourages the shared backbone to learn features that are useful both for detailed object categorization and for the higher-level decision of whether an object is a valid target at all.

The backbone of the network consists of several convolutional blocks. Each block includes a convolutional layer, batch normalization, a rectified linear unit (ReLU) activation and a max-pooling layer. The number of feature channels increases in deeper layers (e.g., from 32 to 64 to 128) while spatial resolution is gradually reduced. At the end of the convolutional stack, global average pooling collapses the spatial dimensions into a fixed-length feature vector. This vector summarizes the visual information present in the image in a way that is convenient for classification.

From this shared feature representation, two separate fully connected heads are constructed:

- **Fine head** – feeds the features through one or more dense layers and ends with a softmax layer over N fine classes, corresponding to all canonical categories defined in the unified dataset.
- **Coarse head** – branches off from the same backbone features and ends with a softmax layer over three classes: `invalid_background`, `invalid_nontarget` and `valid`.

During training, both heads are optimized simultaneously using categorical cross-entropy losses, which can be combined with different weights if one task is considered more important than the other. To improve generalization, the model incorporates several regularization techniques. Dropout is applied after dense layers to reduce overfitting by randomly deactivating a fraction of neurons during training. L2 weight regularization can be applied to convolutional and dense kernels to penalize excessively large weights. The model is optimized using Adam or AdamW, with a configurable learning rate and weight decay parameter. When training on a GPU, mixed-precision training can be enabled so that most computations use lower-precision floating point formats, improving training speed and memory efficiency without sacrificing accuracy.

4. Training Methodology

The training script, implemented in TensorFlow, reads `metadata.csv` and constructs appropriate `tf.data` pipelines for the training, validation and test splits. For each row of the metadata file, the script loads the image from the `images` directory using the `out_relpath` field, decodes it into an RGB tensor, resizes it to the fixed input resolution of the network and normalizes pixel values to a standard range. Data augmentation operations such as random horizontal flips, small

rotations and brightness or contrast adjustments can be applied to improve robustness, provided they do not destroy small targets or make labels ambiguous.

The labels for each example are derived from the `fine_label` and `coarse_label` fields. These textual labels are mapped to integer indices using the label map. The training pipeline can compute sample weights to address class imbalance, down-weighting over-represented classes such as `invalid_background` and up-weighting under-represented valid classes. An optional setting determines whether invalid examples should contribute to the fine-class loss. When this setting is disabled, invalid images are used primarily to train the coarse head, while the fine head focuses on examples that have meaningful fine labels. In one configuration, the network is trained end-to-end from scratch, with both heads active from the beginning. In another configuration, the coarse head is warmed up first to learn a reliable separation between valid and invalid examples, after which the fine head is trained jointly with the coarse head. A learning rate scheduler reduces the learning rate when the validation metrics plateau, allowing the optimizer to fine-tune the model around promising regions of parameter space.

5. Results and Error Analysis

Baseline

To provide a point of comparison, a simple baseline was evaluated in which all images are classified into a single class (the most common fine label) or a single coarse category. The baseline achieved a fine-head accuracy of 0.235 and coarse-head accuracy of 0.345, showing that naïve strategies are not useful for this problem.

CNN performance

The trained multi-output CNN delivers a substantial improvement over the baseline. On the held-out test set of 27,362 samples (11 fine classes), the fine head achieved an overall accuracy of 87.08 %, while the coarse head achieved 91.31 % accuracy. Other important metrics include:

- **Fine-head precision/recall/F1 (macro averaged)** – 0.857 / 0.821 / 0.836.
- **Fine-head per-class precisions** – for example, `civilian_vehicle` (95.38 %), `tank_av` (94.26 %) and `aerial_landscape` (93.06 %) are easiest; `soldier` (77.30 %), `rock_debris` (75.47 %) and `random_animal` (80.27 %) remain more challenging.
- **Coarse-head precision/recall/F1 (weighted)** – 0.9133 / 0.9131 / 0.9132.
- **Coarse per-class precisions** – `invalid_nontarget`: 92.77 %, `invalid_background`: 89.41 %, and `valid`: 91.65 %.

These results indicate that the network learns to separate valid targets from other objects with high precision and recall, and it remains effective at distinguishing among the 11 fine categories.

The confusion matrices show that most errors arise from visually similar categories; for example, the model sometimes confuses soldier with civilian or random_animal with bird.

Impact of input resolution

Several input resolutions were examined during training. Increasing the resolution from 64×64 to 128×128 significantly improved both fine- and coarse-level accuracy, but gains diminished at higher resolutions (e.g., 256×256). In other words, model performance plateaued beyond 128×128. The larger inputs improved classification of subtle features such as distinguishing a tank from a civilian vehicle; however, the additional computational cost was not justified by a corresponding increase in accuracy. Conversely, the real-time video analysis (YOLO + CNN pipeline) benefitted from lower-resolution crops. When using smaller input resolutions, the system still accurately detected distant or low-focus objects, and the reduced computational load allowed the video pipeline to operate closer to real time. Thus, a trade-off exists between classification accuracy and real-time performance: for offline classification tasks, higher resolution (up to 128×128) is beneficial, whereas for live video, lower resolutions suffice and may even improve detection of distant targets due to the larger effective receptive field.

Error patterns

The most common failure modes observed during evaluation include:

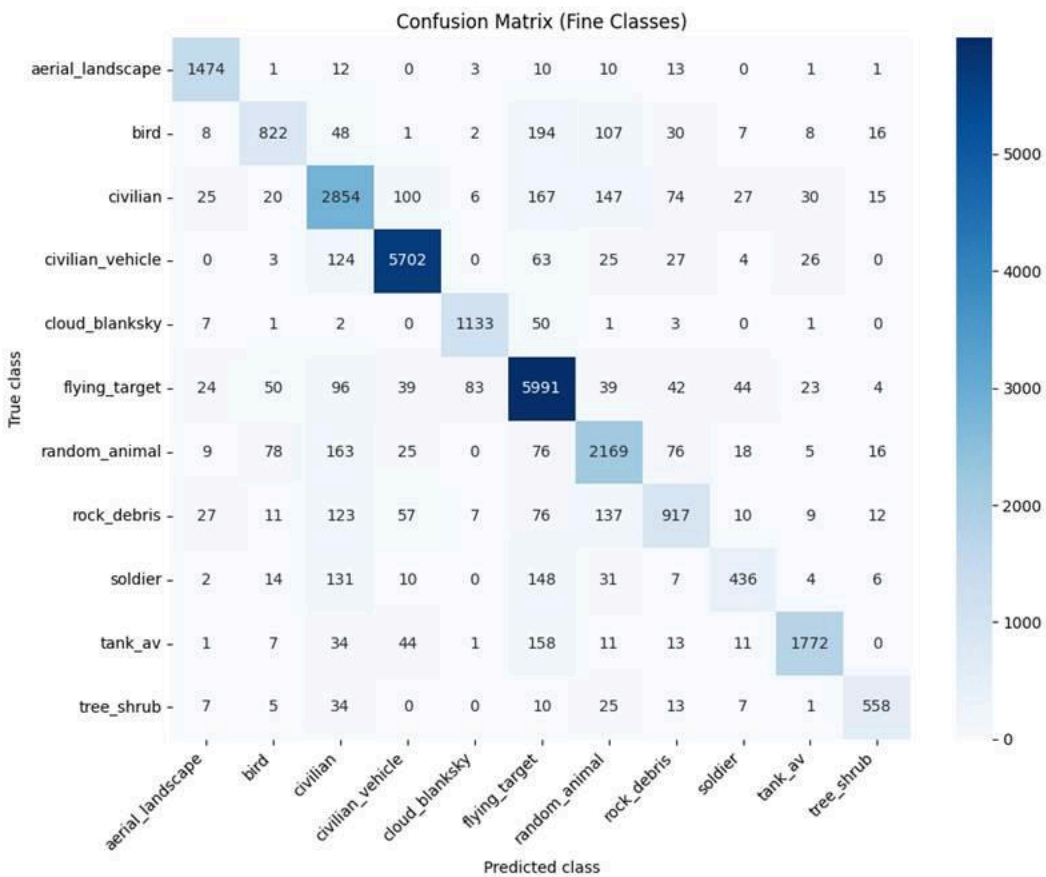
- **Borderline crops** – bounding boxes or segmentation masks sometimes include a mixture of target and background pixels. If the crop is dominated by background, the classifier may assign it to an invalid category even when part of a target is visible.
- **Small or occluded targets** – small or heavily occluded objects provide too little visual information, reducing confidence in the predicted label.
- **Ambiguities between similar classes** – classes such as civilian vs. soldier or bird vs. random_animal share similar appearance and can confuse the fine head.

Confusion matrices computed for both the coarse and fine heads highlight which pairs of classes are most frequently confused. For example, the model sometimes predicts invalid_nontarget when the true label is invalid_background and vice versa. Collections of misclassified images, grouped by true and predicted labels, provide qualitative insight into the kinds of scenes that challenge the model.

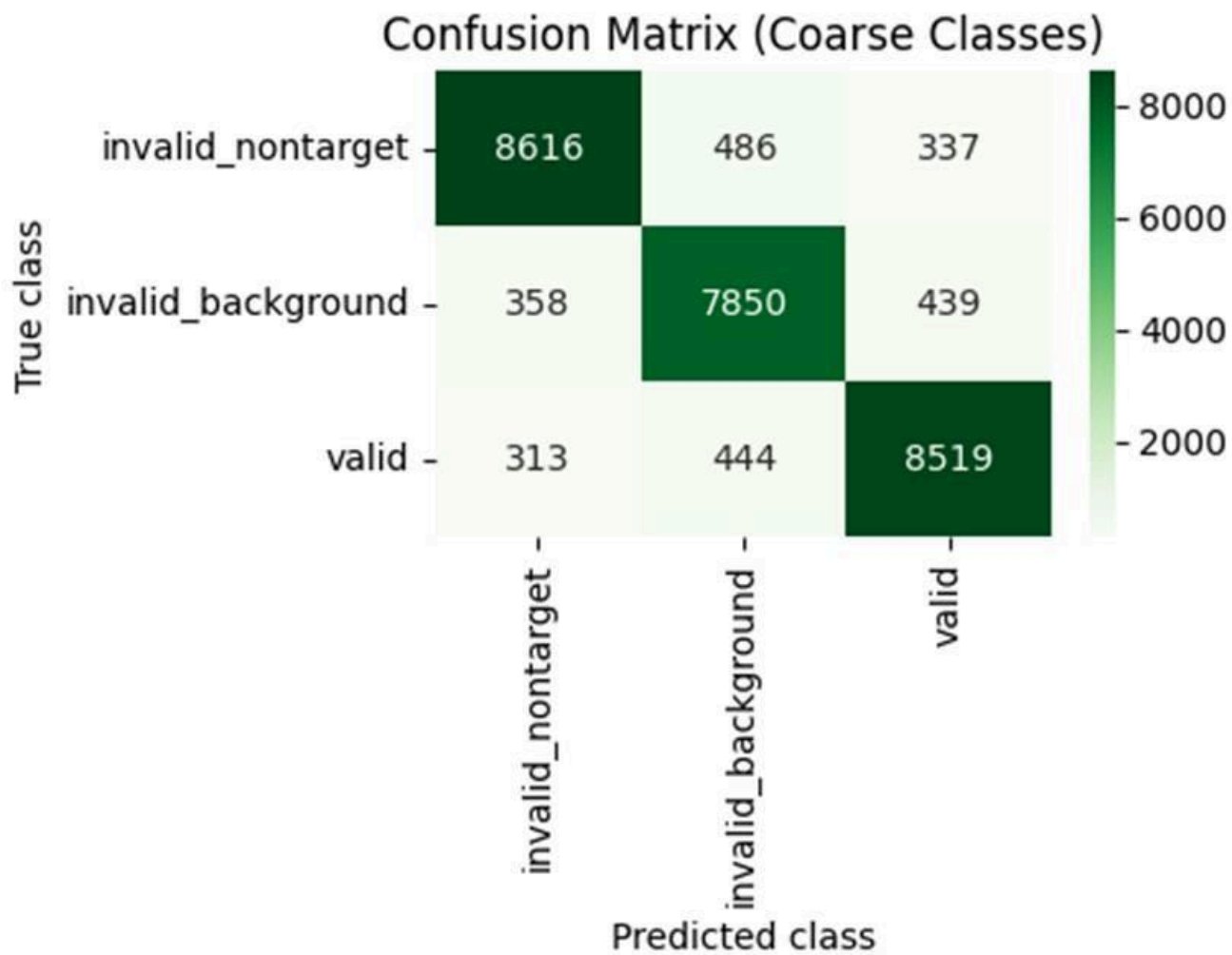
Visualizing the results

To help interpret the test outcomes, confusion matrices and per-class precision plots were generated from the evaluation data.

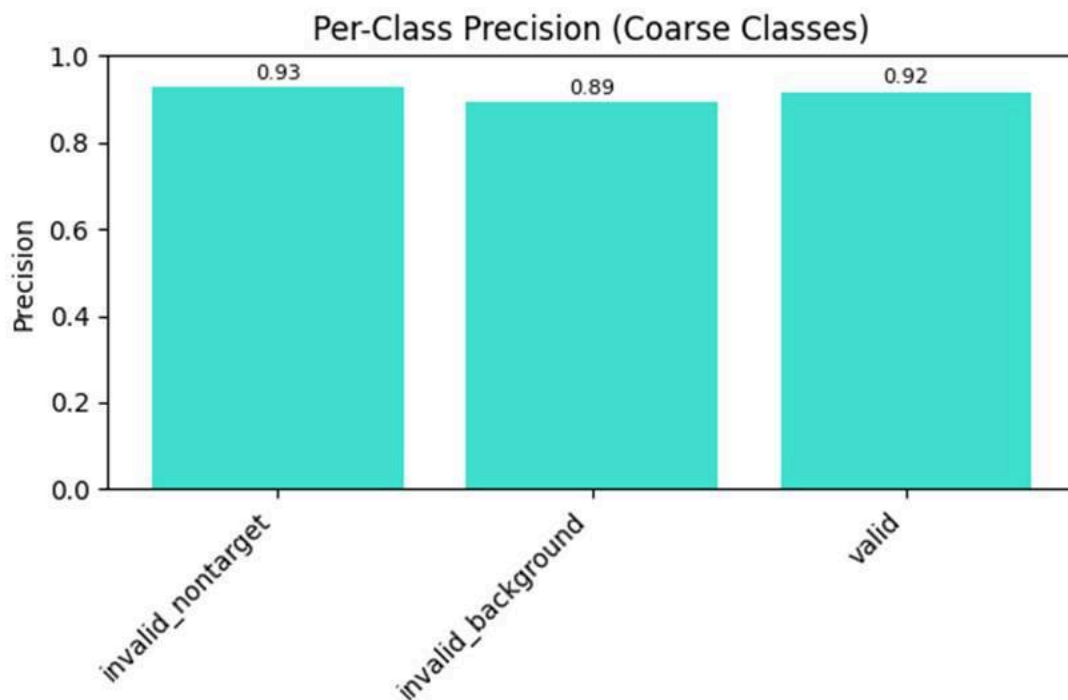
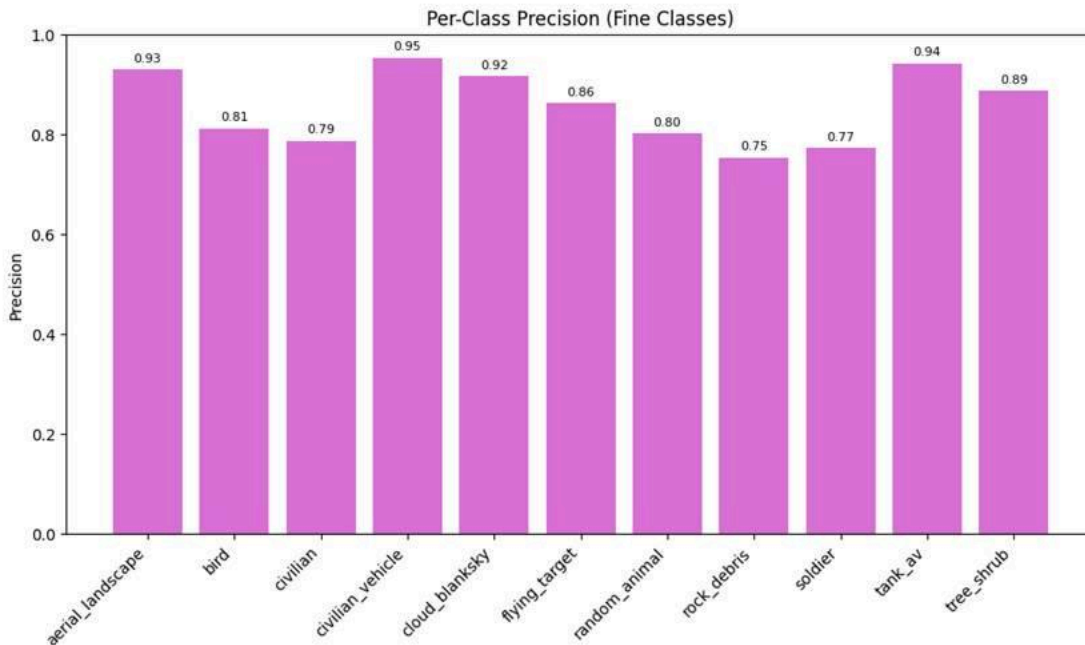
Fine confusion matrix: The heatmap below illustrates the 11-class confusion matrix. The color intensity reflects the number of test samples in each true/predicted pair (darker cells indicate more samples). The diagonal dominance confirms the high accuracy of the fine classifier, whereas off-diagonal entries highlight specific confusions (e.g., *soldier* vs. *civilian*).



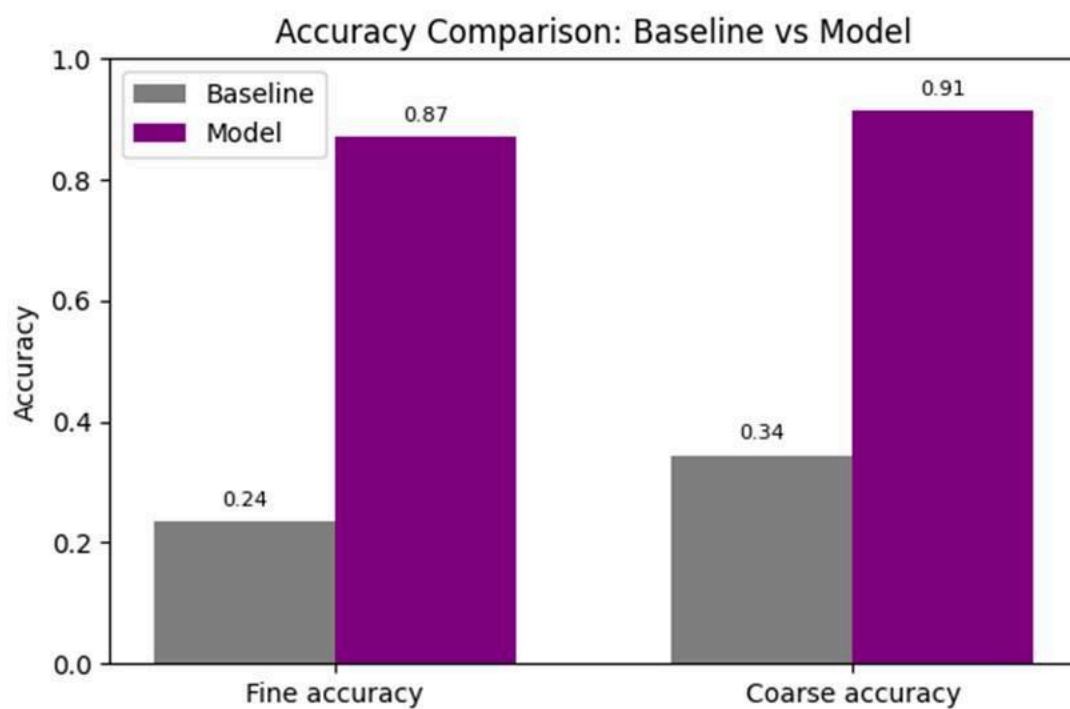
Coarse confusion matrix: The following heatmap summarizes the three-way coarse classification results. Most examples fall on the diagonal, indicating that the coarse head reliably separates valid targets from nontargets and background.



Per-class precision: Bar charts below show the test precision for each fine category (left) and each coarse category (right). The fine classifier performs best on civilian_vehicle, tank_av and aerial_landscape, while classes such as soldier, rock_debris and random_animal remain more challenging. The coarse head shows uniformly high precision across its three categories.



Model vs. baseline accuracy: The bar chart below compares the test accuracy of the trained model with the naïve baseline. The CNN dramatically outperforms the baseline on both fine and coarse tasks.



Per-class precision tables:

Fine classes:

Fine class	Precision
aerial_landscape	0.9306
bird	0.8123
civilian	0.7882
civilian_vehicle	0.9538
cloud_blanksky	0.9174
flying_target	0.8629
random_animal	0.8027
rock_debris	0.7547
soldier	0.7730
tank_av	0.9426
tree_shrub	0.8885

Coarse classes:

Coarse class	Precision
invalid_nontarget	0.9277
invalid_background	0.8941
valid	0.9165

6. Real-Time Utilization Program

The final component of the project is a utilization program that integrates the trained classifier into a real-time detection pipeline. This program is intended to run on video streams from a webcam, a video file or a live drone feed (testing to date has used solely video files). It uses a YOLO segmentation model, configured for instance segmentation, to locate candidate objects in each frame. For each segmentation mask produced by YOLO, the program extracts a crop from the original frame and resizes it to the classifier's input resolution. The same normalization procedure used during training is applied so that the classifier receives inputs with consistent statistics.

The cropped images are then passed through the CNN classifier. In the simplest real-time configuration, only the coarse head is used in the live display, because the primary question is whether a detected object is a valid target, a non-target object or pure background. Based on the predicted coarse label, the program overlays visual annotations on the original frame:

- Regions classified as `invalid_background` are ignored and receive no overlay.
- Regions classified as `invalid_nontarget` are marked with one color (e.g., red) to indicate that they contain real objects that are not targets.
- Regions classified as `valid` are marked with another color (e.g., green) and can optionally be labeled with the fine-class prediction.

The resulting frame is displayed along with an estimate of the frame rate. The code can optionally skip frames while processing to maintain real-time performance. This modular design separates the responsibilities of detection (YOLO) and classification (CNN), allowing either component to be upgraded independently. For example, a newer YOLO model could be substituted for better localization performance, or the classifier could be updated when new fine classes are added to the label taxonomy. 📺 `overlay_coarse(V6)(32x32).mp4`

📺 `overlay_coarse(V3)(32x32).mp4` , show preprocessed implementation of the pipeline.

7. Discussion and Future Work

This project demonstrates that a unified dataset builder, a multi-output CNN classifier and a YOLO-based utilization program can be combined into a coherent pipeline for drone- and camera-based target identification. Several practical lessons emerge:

1. **Label consistency and dataset hygiene** – Aliasing different terms to canonical labels and separating fine classes from coarse validity categories prevents confusion during training and evaluation. Careful handling of missing or corrupted images and removal of duplicates is essential.
2. **Preprocessing consistency** – Matching the resizing, normalization and color-space transformations between training and deployment is essential for reliable performance. Any mismatch can degrade accuracy in real-time use.
3. **Resolution trade-offs** – Higher-resolution inputs improve fine-grained accuracy up to about 128×128 , after which returns diminish, while lower-resolution inputs suffice for real-time detection and may even improve identification of distant targets.

The current system has limitations. Performance is heavily dependent on the diversity and representativeness of the training data: if the unified dataset does not adequately cover different drone altitudes, camera angles, lighting conditions or environmental contexts, the model may generalize poorly. Small and distant targets remain challenging because they occupy relatively few pixels. The label taxonomy is static; adding new fine classes requires retraining or at least modifying and fine-tuning the output layers. Several directions for future work are promising:

- **Temporal information** – Use multiple frames or short video clips rather than treating each frame independently, allowing the system to exploit motion cues.
- **Backbone architectures** – Experiment with backbones pretrained on aerial imagery or large-scale detection datasets.
- **Active learning** – During real runs, log uncertain or misclassified detections, manually label them and add them to the training set.
- **Lightweight models** – Explore more efficient architectures (e.g., MobileNet variants) for deployment on constrained hardware while maintaining accuracy.

8. Conclusion

In summary, this project develops a complete deep learning pipeline for drone- and camera-based target identification. The dataset builder consolidates heterogeneous public datasets into a single, labeled corpus suitable for training. The multi-output CNN architecture learns to predict both fine-grained classes and coarse validity categories, enabling the system to distinguish valid targets from non-targets and background. Testing shows that the CNN achieves 87 % fine-classification accuracy and 91 % coarse-classification accuracy, far surpassing the baseline. The YOLO-based utilization program demonstrates how the trained classifier can be embedded into a real-time application that operates on live video streams. Although there is substantial room for improvement through additional data, refined architectures and temporal modeling, the work establishes a flexible and modular foundation for future research and application in aerial and surveillance-based target recognition.

For Reference:

https://drive.google.com/drive/folders/1z6jWKO7EuR7loUj6pzNLlkax223k7bp2?usp=drive_link
<https://www.kaggle.com/datasets/loggger/fpv-images>